COMPUTER SCIENCE · 6B06101

# Automated Tree Recognition & Green-Space Mapping

Turning a manual, one-by-one task into an **automated** process — from a satellite image to a city tree-map, end to end, for Astana.

AUTHORS Berik Sharipov · Anuar Totin · **Rasul Aidarkhanov** SUPERVISOR S. Satbayev

AGENDA

# What we'll cover

01 **Problem & relevance** — why Astana needs this

02 **The gap & aim** — off-the-shelf scores ~0 here

03 **Objectives, problem statement & methods**

04 **Literature review**

05 **How we measure success** (the maths)

06 **The automated pipeline** (architecture)

07 **Data, database & hardware**

08 **Four detection engines** — YOLO · Mask R-CNN · DeepForest · SAM 2

09 **Why several models** & ensembles

10 **Results** on one common test set

11 **Canopy** — the web application

12 **Discussion & conclusions**

RELEVANCE

# Mapping a city's trees — today it's done by hand

Astana is expanding fast. Urban green cover matters for **heat mitigation, irrigation planning and green-space policy** — but to know where the trees are, someone has to **count them one by one** off imagery. That doesn't scale to a whole city.

The real problem isn't "can we beat a benchmark" — it's that **there is no automated process** for this in Astana at all. Existing models are trained on North-American / European data, and we didn't even know if they work here.

So our goal was to **automate the whole process** end-to-end — and to be honest about how well it works.

**TODAY — MANUAL**

Counting trees by eye, image by image. Slow, not repeatable, never finished for a growing city.

**OUR GOAL — AUTOMATED**

One repeatable pipeline: point at an area → it finds the trees and puts them on a map. No manual counting.

WHY WE COULDN'T JUST USE AN EXISTING TOOL

# 0.012

Box mAP@50

A popular ready-made detector (NEON DeepForest), run as-is on Astana — essentially blind. There was **no off-the-shelf automation** that worked here, so we had to build our own.

AIM

Build an **automated end-to-end system** that recognises trees and maps green space on Astana satellite imagery — and deliver it as a working tool, being honest that the accuracy is modest.

OBJECTIVES

# Five tasks

1. Build an **annotated Astana dataset** (tiled satellite imagery, instance polygons).
2. Train and compare four **detection engines** — YOLOv8-seg, Mask R-CNN, DeepForest, SAM 2.
3. **Combine** them through ensembles and test if that helps.
4. Evaluate everything on **one common test set** for a fair comparison.
5. Wrap it all in an **automated web application** — the actual deliverable.

METHODS

Supervised deep learning · transfer learning from pretrained weights · sliding-window tiling of large images · mAP@50 as the single comparison metric.

FORMALLY — WHAT GOES IN, WHAT COMES OUT

# Input → Output

### INPUT

A satellite capture of an area of Astana at zoom 19 (≈ 0.3 m per pixel), cut into overlapping **~640 px RGB tiles** by a sliding window.

Training input also carries human **instance-polygon annotations** — one outline per tree.

### OUTPUT

For every detected tree: a **bounding box**, an **instance mask** and a **confidence score**.

Tiles are stitched back, duplicates merged, and the result is shown on a city **map** with per-area counts and **GeoJSON / CSV** export.

$$\text{tiles (RGB)} \;\rightarrow\; \text{model } f(\cdot) \;\rightarrow\; \{\ \text{box, mask, confidence}\ \}_{\text{per tree}} \;\rightarrow\; \text{merge \& map}$$

LITERATURE REVIEW · 31 PAPERS · 2019–2025

# What others report — and on what data

| METHOD | AUTHORS · YEAR | DATA | BEST REPORTED METRIC |
|---|---|---|---|
| **Mask R-CNN (MCAN)** | Lv et al. · 2023 | UAV RGB | Detection AP 92.4% |
| **Faster R-CNN / RetinaNet** | dos Santos et al. · 2019 | UAV RGB | AP 82–93% |
| **YOLOv12m** | Abbas & Damaševičius · 2025 | RGB satellite | mAP@50 90.8% |
| **DeepLabV3+ / U-Net** | Martins / Wang · 2021 | Aerial 10–32 cm | F1 91% · IoU 96% |
| **DeepForest (off→fine)** | Ventura et al. · 2024 | NAIP 60 cm (USA) | F 0.42 → 0.73 |
| **SAM / SAM 2** | Kirillov / Ravi · 2023–24 | 1B+ masks | zero-shot segmentation |
| **YOLOv8x-seg (ours)** | **This work · 2026** | **Astana satellite · M14** | Box mAP@50 0.315 |

High accuracy — but always on the authors' own region and sensor. **Our row is the first measurement of these models on Astana imagery** (first, not best).

THE MATHS, IN PLAIN TERMS

# Three simple ideas — one comparison number

### 1 · IOU — DID THE BOX LAND ON THE TREE?

$$\text{IoU} = \frac{\text{overlap area } (A \cap B)}{\text{combined area } (A \cup B)}$$

How much a predicted box overlaps the real one. **1.0** = perfect, **0** = miss. A tree counts as found when **IoU ≥ 0.5**.

### 2 · PRECISION & RECALL

**Precision** — of the trees we flagged, how many were real (few false alarms).
**Recall** — of all real trees, how many we found: **ours ≈ 30%** at the default setting.

### 3 · MAP@50 — THE SINGLE SCORE

Sweep the confidence threshold, track precision against recall, and take the **area under that curve** (at IoU ≥ 0.5). One honest number to compare every model the same way — it is the **0.315** on the results slide.

### HOW THE TWO DETECTORS WORK

**YOLOv8** — one pass: splits the image into a grid and predicts boxes + masks in a single look (fast).
**Mask R-CNN** — two passes: first proposes regions, then refines them (slower, careful).

THE AUTOMATED PIPELINE — THE CORE OF THE PROJECT

# From a satellite image to a tree-map — automatically

**FRONT END**
React 18 + Leaflet
pick an area & a model

→

**API · FASTAPI**
tile (sliding window)
→ inference → merge

→

**ENGINE (SWAPPABLE)**
YOLOv8-seg · Mask R-CNN
DeepForest · Ensemble

→

**STORE · SQLITE**
scans · snapshots
runs · detections

→

**MAP & EXPORT**
Leaflet layers
GeoJSON · CSV · HTML

### TILING

Large captures are cut into ~640 px tiles with overlap (zoom 19, ≈0.3 m/px); detections are stitched back and de-duplicated.

### SWAPPABLE ENGINE

The detection model is just one plug-in step — swap YOLO for Mask R-CNN, DeepForest or an ensemble without touching the rest of the pipeline. ~1 km² at zoom 19 in ≈18 s on a laptop GPU.

INPUTS & INFRASTRUCTURE

# What goes in, and where it runs

## DATASET

High-zoom satellite captures of Astana (~0.3 m/px), tiled 640 + 128 overlap. Trees overlap, so we annotate **instance polygons**, not just boxes. A held-out common set of **14 images · 702 labelled trees** scores every model.

## DATABASE (SQLITE)

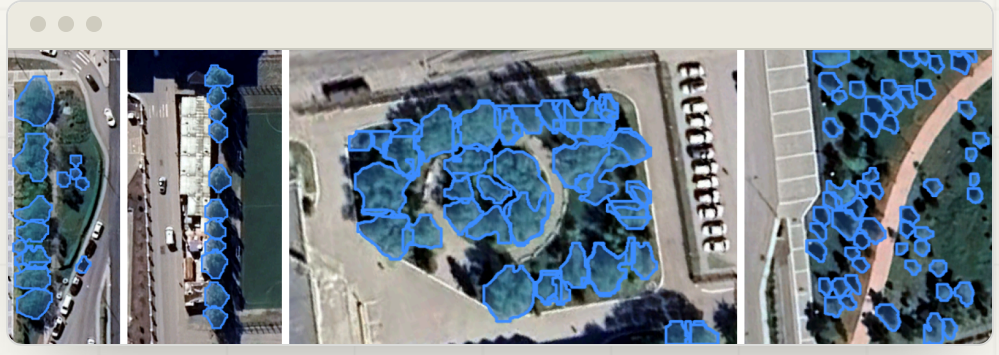
SCAN_SESSIONS → SNAPSHOTS → RUNS → DETECTIONS

One-to-many, **ON DELETE CASCADE**.

## HARDWARE

- RTX 4060 8 GB — YOLO + all common-set inference
- RTX 4070 — Mask R-CNN
- RTX 4050 — DeepForest

8 GB VRAM ceiling → batch 2 + mixed precision. Deliberately modest, to prove reproducibility without a cluster.



FOUR INTERCHANGEABLE ENGINES · ONE PIPELINE

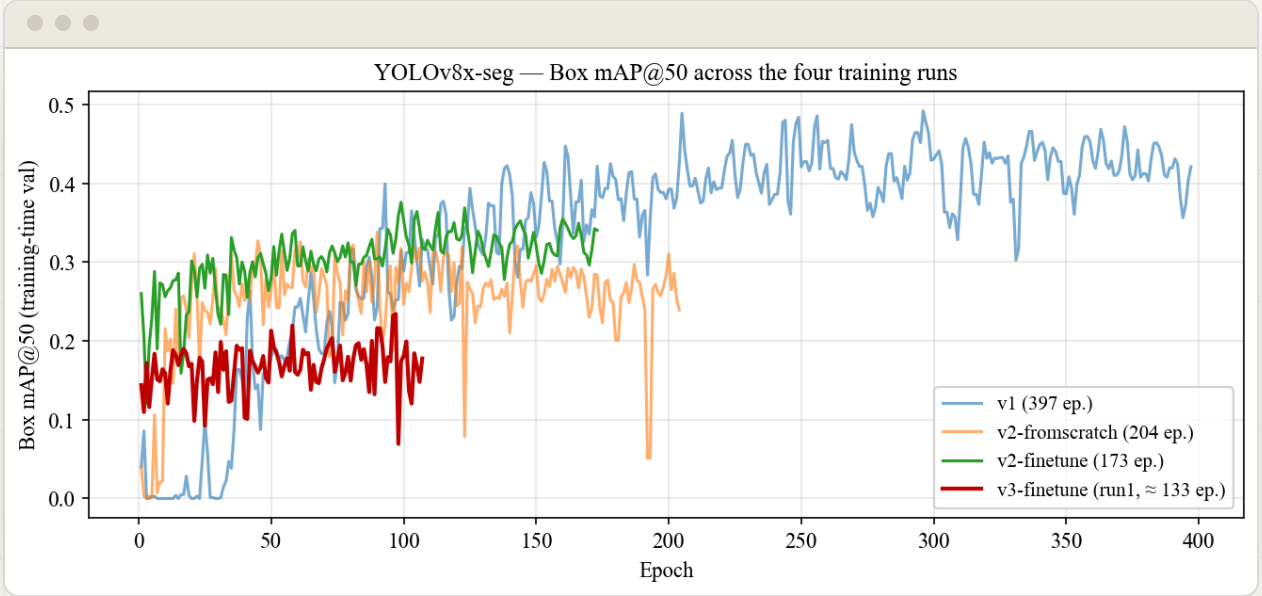
# The automation doesn't depend on one model — any of four can drive it.

YOLOv8-seg (Rasul) · Mask R-CNN (Berik) · DeepForest + SAM 2 (Anuar) · plus two ensembles. Each plugs into the same automated pipeline and is scored on the same 14 images.

MY BRANCH · ~20 EXPERIMENTS

MY CONTRIBUTION

# YOLOv8x-seg: from 0.131 to 0.315




Box mAP@50 across every experiment on the common set.

# +140%

Box mAP@50 over my own first model (0.131 → **0.315**).

### WHAT ACTUALLY MATTERED

1. Start from pretrained (COCO) weights
2. Tile at the resolution you deploy at
3. Keep augmentation moderate

### WHAT I ALSO FOUND

Default augmentation beat hand-tuned (top-down trees are rotation-invariant); model size is **U-shaped** (m & x best, l worse); run-to-run noise ≈ ±0.03 — so small gaps aren't real.

ENGINE #2 · BERIK SHARIPOV

# Mask R-CNN — the careful, mask-first engine

A classic **two-stage, region-based** detector. Where YOLO looks once, Mask R-CNN looks twice — propose, then refine — which makes it slower but gives it naturally **crisp instance masks**. In our project it is one of the swappable engines behind the same automated pipeline.

## HOW IT WORKS

A **Region Proposal Network** scans each tile and suggests candidate boxes → **RoIAlign** crops every region → three parallel heads output a **box**, a **class** and a per-pixel **mask**. Segmentation is built in, not bolted on, so outlines hug the crown.

## HOW BERIK TRAINED IT

Transfer-learned from a **COCO-pretrained** Mask R-CNN and fine-tuned on the same annotated Astana tiles (v2 + v3) for a single **"tree"** class — same train/val split as every other engine, so the comparison is fair. (Exact backbone/epochs to confirm with Berik.)

## HOW IT DOES ON ASTANA

Box mAP@50 **0.166**, Mask **0.158**. At confidence 0.5: **precision 0.44 · recall 0.22** (157 of 702 trees). Strongest on medium, well-separated crowns; like the others, it loses the small and shaded ones.

## WHY IT STAYS IN THE APP

Overall below YOLO on our data — but it recovers some **larger trees the others miss**, and its masks are the cleanest. That is exactly why the pipeline keeps it as a user-selectable engine rather than throwing it away.

ENGINE #3 · ANUAR TOTIN

# DeepForest + SAM 2 — the forest specialist, re-taught

A two-part engine: **DeepForest** (a tree-crown detector, RetinaNet-style) finds the trees, then **SAM 2** (Segment Anything 2) turns each box into a high-quality mask. It is the engine that produced the **0.012** that started the whole project — and the clearest proof of why domain adaptation matters.

### HOW IT WORKS

**DeepForest** outputs a box per crown; each box becomes a **prompt** for SAM 2, which segments the exact pixels. Two specialist models chained — a detector that knows "tree", and a segmenter that knows "object".

### THE DOMAIN PROBLEM

DeepForest is pretrained on **North-American airborne forest** imagery. Pointed at a dry steppe city it was nearly blind — **0.012**. Not a bug, a mismatch: the trees, spacing and sensor all differ from what it had ever seen.

### HOW ANUAR FIXED IT

Fine-tuned DeepForest on the annotated Astana data and paired it with SAM 2 masks (confidence 0.3, 400 px patches). Result: **0.012 → 0.146** Box mAP@50 (Mask **0.134**) — roughly a **×12** jump from re-teaching alone.

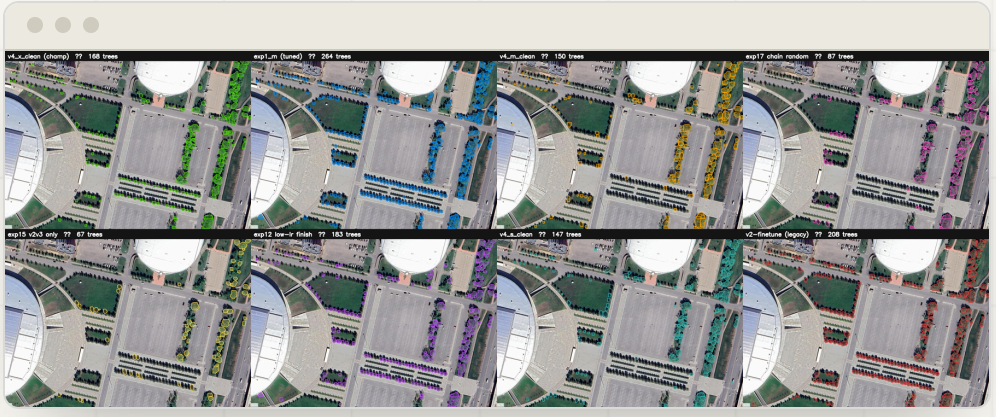
### WHY IT STAYS IN THE APP

SAM 2 gives the **best general-purpose masks** of any engine, and the off-the-shelf→fine-tuned story is the project's core lesson. It still struggles with tiny, dense urban trees — so it's offered alongside the others, not instead of them.

THE MOST USEFUL LESSON

# No single model wins everywhere

Each model catches **some trees the others miss**. So instead of forcing one "winner", we kept all of them in the app and let the user **choose the model** — and we tried combining them.



TWO ENSEMBLES

A **Weighted Box Fusion** across model families

B **Cross-YOLO 4-vote** — kills single-model hallucinations (e.g. stadium-roof false positives)

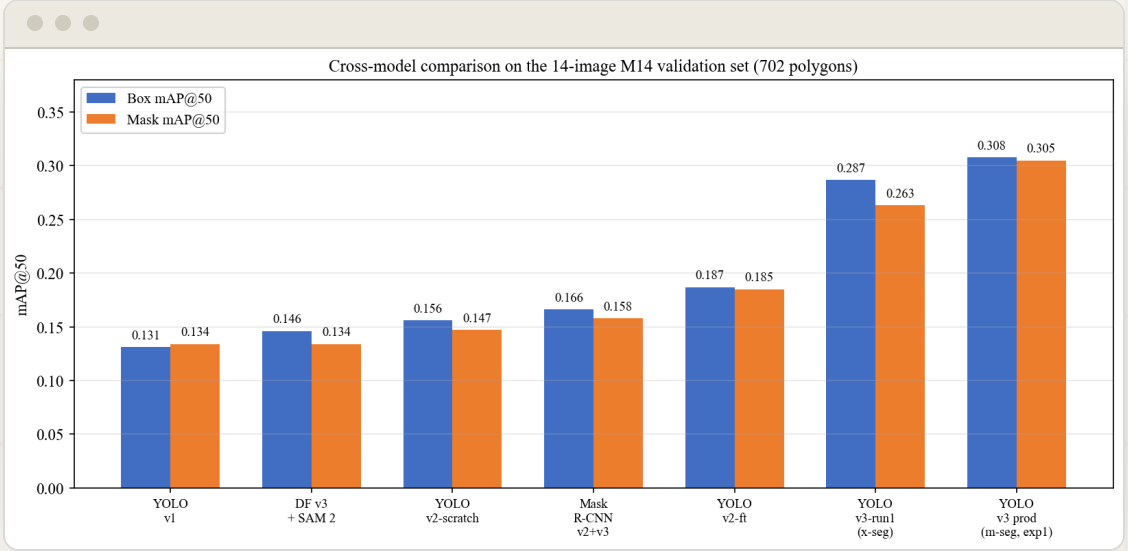
More robust on hard tiles. By the numbers the single YOLO still scores highest on our set — but that doesn't make it right for every image.

HEAD-TO-HEAD · SAME 14 IMAGES · 702 TREES

# One common test set, every model

| MODEL | BOX MAP@50 | MASK MAP@50 |
|---|---|---|
| **YOLOv8x-seg v4 (ours)** | 0.315 | 0.289 |
| YOLOv8x-seg v3 | 0.287 | 0.263 |
| Mask R-CNN | 0.166 | 0.158 |
| DeepForest + SAM 2 | 0.146 | 0.134 |
| YOLO v1 baseline | 0.131 | 0.134 |
| NEON DeepForest (off-the-shelf) | 0.012 | — |

Champion operating point (conf 0.25): **precision 0.52 · recall 0.29** (222 of 755 trees). Margins between top models are small — partly within run-to-run noise.

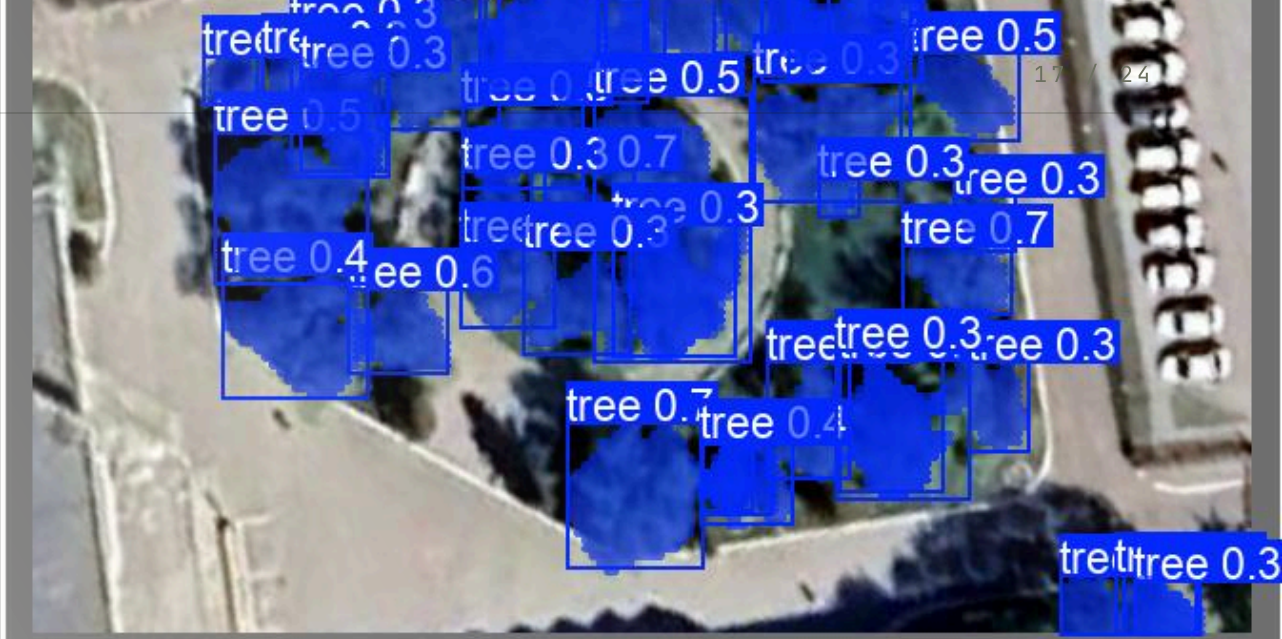



We read this as "these engines are in the same ballpark on Astana," **not** "one clearly wins." What matters for the project: **any of them can drive the automated pipeline.**

WHAT IT ACTUALLY LOOKS LIKE

# Detections vs. real crowns

Predictions line up with real tree crowns in many places. But honestly — it **doesn't find every tree**: at the app's default confidence it finds about **30%** of labelled trees (more if looser, fewer if stricter), and per image it swings from almost none to nearly all.

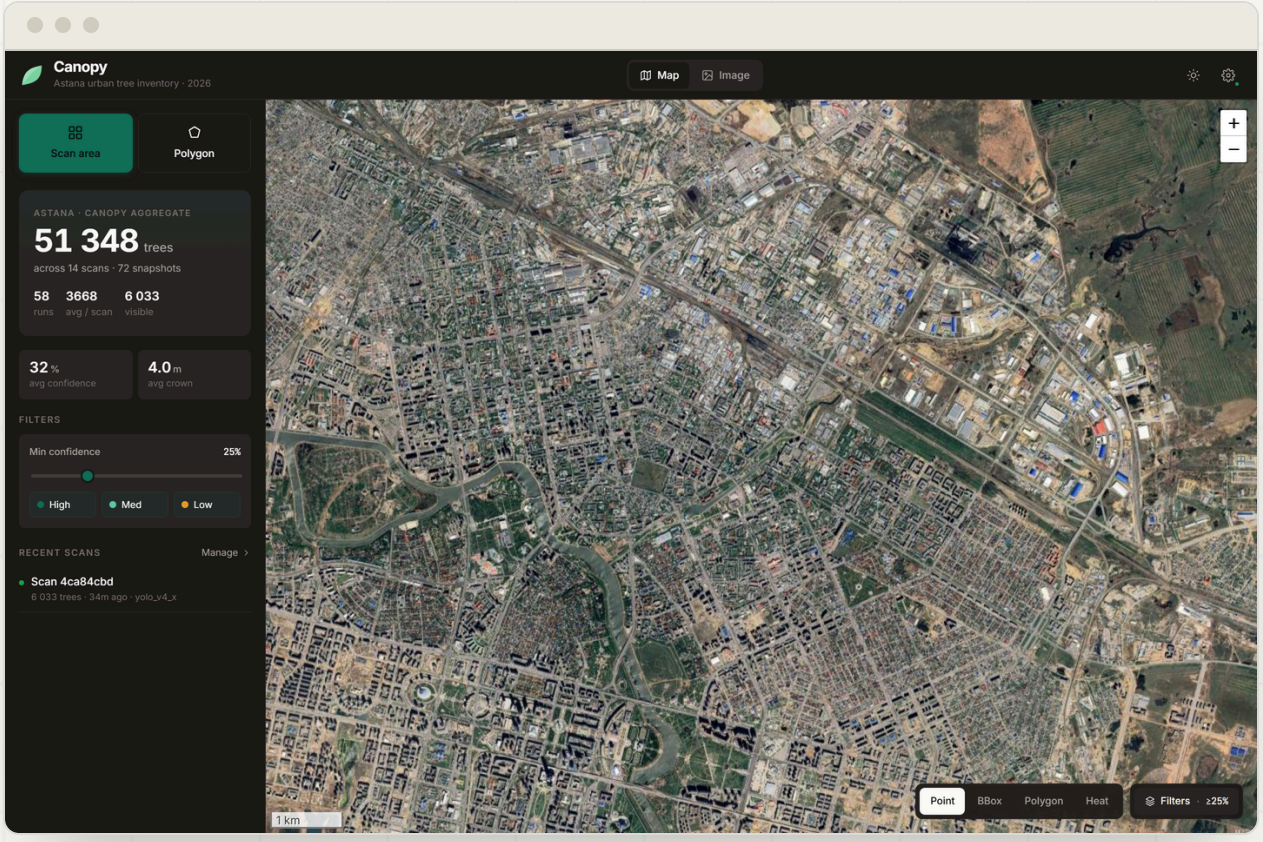


HONEST ABOUT MISSES — SMALL & SHADED TREES

THE DELIVERABLE

MY CONTRIBUTION

# Canopy — pick an area, pick a model, scan



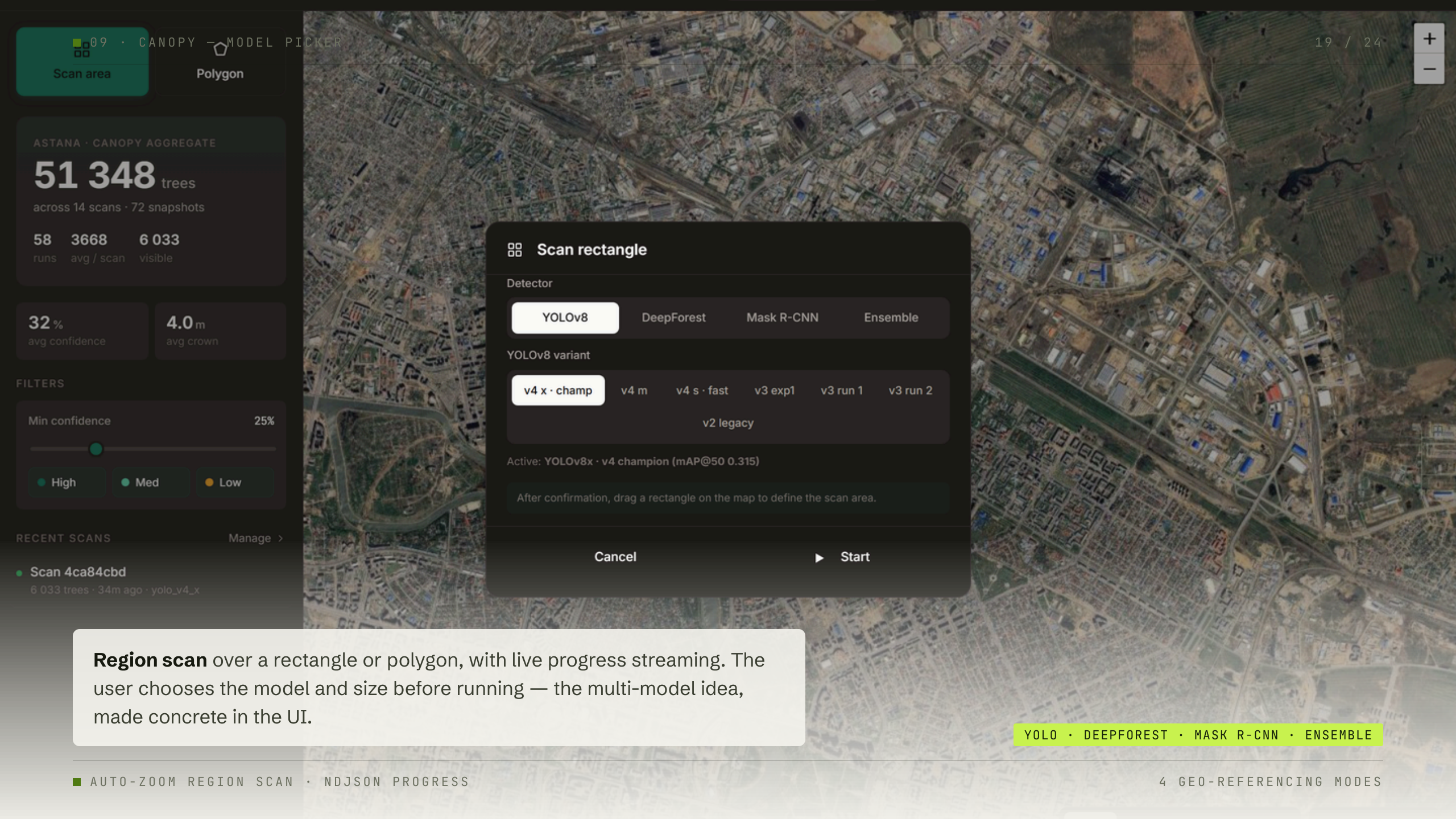

### ONE FLOW

Select an area of Astana → the app tiles it, runs the chosen model, and draws every detected tree on the map — with a confidence filter and Point / Box / Polygon / Heat layers.

### FREE CHOICE OF MODEL

YOLOv8 (7 sizes) · DeepForest ± SAM 2 · Mask R-CNN · Ensemble — because different models suit different images.

Counts shown in screenshots come from repeated demo scans — **not** a full city census.



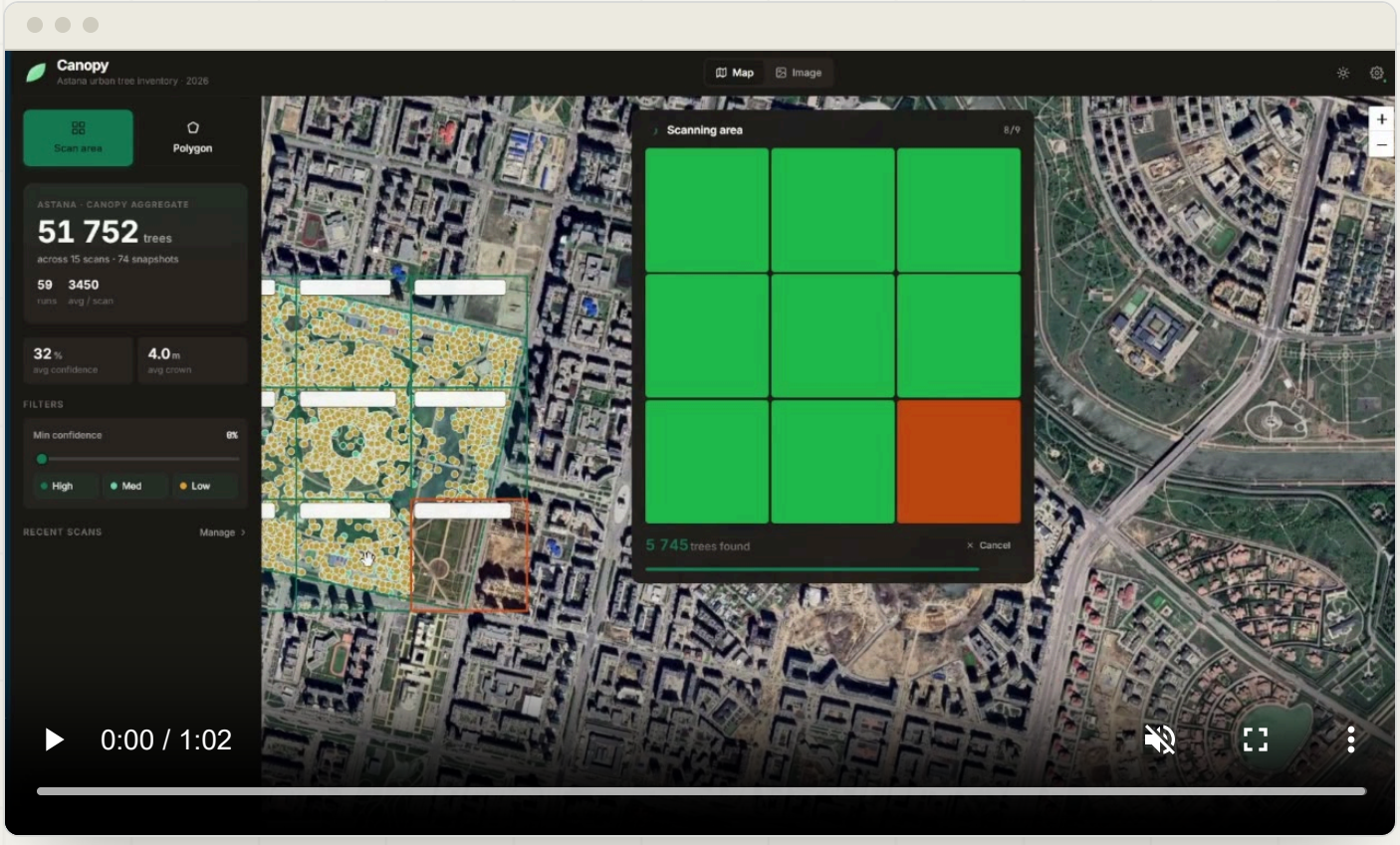

**Region scan** over a rectangle or polygon, with live progress streaming. The user chooses the model and size before running — the multi-model idea, made concrete in the UI.

YOLO · DEEPFOREST · MASK R-CNN · ENSEMBLE

THE PIPELINE RUNNING END-TO-END

MY CONTRIBUTION

# Live demo — automated scan of the Botanical Garden




## WHAT YOU'RE SEEING

The v4 champion (YOLOv8x-seg) scanning Astana's Botanical Garden end-to-end in Canopy: **tile** → **detect** → **map**, with every detected tree drawn live.

## WHY THIS SCENE

A dense, well-defined planting — the clearest case to show the pipeline working. Honest reminder: on sparser, harder scenes recall drops (see the qualitative slide).

▶ Plays automatically on this slide · ~1 min · click for controls.

HONEST ASSESSMENT

# What we'd say up front

**LIMITATIONS**

- Small training set → modest accuracy (0.315 mAP, ≈30% recall)
- Domain shift: training captures vs. live map tiles
- Misses small & heavily shaded trees

**BUT ALSO**

- Off-the-shelf fails here (0.012) — our models do meaningfully better
- A fair, reproducible comparison on local data
- A working tool, not just a notebook

None of this is hidden — it's the honest state of a student project, and each limit improves with more annotated data.

OBJECTIVES → OUTCOMES

# Every task, addressed

- ✓ **An automated pipeline** — capture → tile → detect → map → export, with no manual counting.
- ✓ **Working application** — Canopy, deployed, with a free choice of detection engine.
- ✓ **Dataset** — annotated, tiled Astana imagery with a held-out 14-image / 702-tree common set.
- ✓ **Four engines compared** + two ensembles, all scored identically on one set.
- ✓ **Fair evaluation** — first such measurement for Astana.

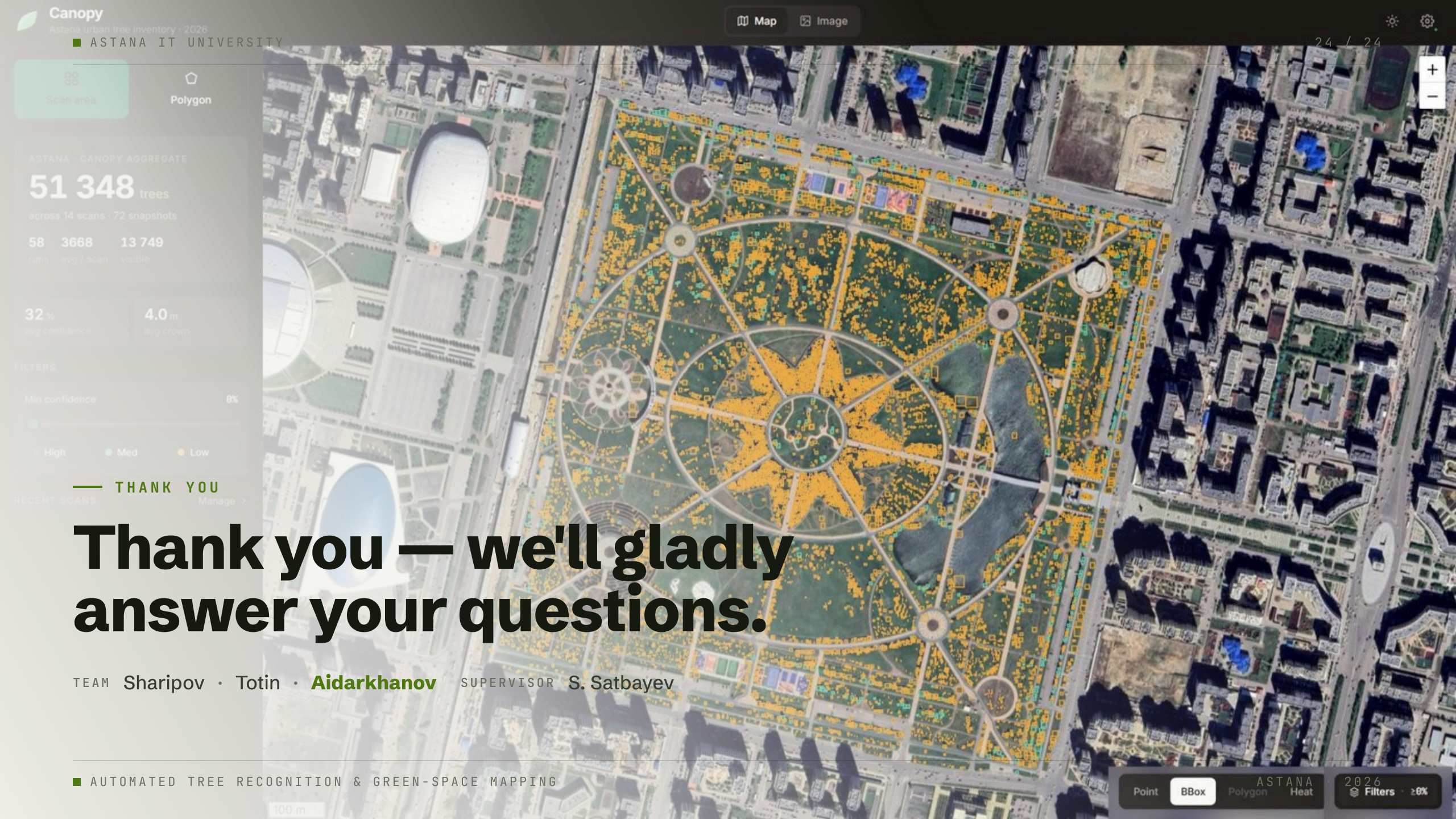
> The headline isn't a score — it's that a **manual task is now an automated process**. Accuracy is modest (and honestly so: ready-made tools score ~0 here, ours do meaningfully better), but the **process** is the contribution.

## MY CONTRIBUTION (RASUL)

1. The whole **automated pipeline & Canopy app** (FastAPI + React + SQLite) — the actual deliverable
2. The **YOLOv8 engine** — ~20 experiments, +140% over my baseline
3. The **cross-YOLO vote** ensemble & the common-set evaluation

## FUTURE WORK

1. More annotated data
2. Train on the **same imagery we run on** (close the domain gap)
3. Better handling of small trees · city-wide canopy analytics




THANK YOU

# Thank you — we'll gladly answer your questions.

TEAM Sharipov · Totin · **Aidarkhanov** SUPERVISOR S. Satbayev